Toward Reliable Automatic Proxies for Code Comprehensibility

Erfan Arvan
New Jersey Institute of Technology
Newark, NJ, USA
ea442@njit.edu

Abstract

Improving code comprehensibility requires measuring it reliably, yet no reliable automatic proxy for comprehensibility exists. Developing one requires a validated human-study baseline to test against. However, the human-study proxies used in the literature (e.g., Likert-scale ratings or response time on output-prediction tasks) have not been systematically validated either, and the construct itself lacks a standard definition. We address this foundational gap first: we introduce a conceptualization of comprehensibility based on expert agreement, construct a ground truth via the Delphi protocol, and evaluate commonly used human-study proxies against it. We find that proxies based on input–output reasoning and response time are particularly reliable, while syntax-based proxies are especially unreliable. With this validated foundation, we propose a controlled human study to test whether code verifiability—the ease with which static verification tools can analyze code—can serve as a reliable automatic proxy for comprehensibility. If verifiability alone explains comprehension difficulty well, it can be used as a standalone automatic proxy; if it only partially explains it, it becomes one signal among several in a combined automatic proxy. Beyond this study, the research agenda includes reassessing prior work in light of proxy reliability, evaluating LLM-based agents as substitutes for human participants, and developing tools that leverage reliable automatic proxies to improve code comprehensibility.

1 Motivation

Code comprehensibility is a fundamental property of software: most maintenance, debugging, refactoring, and code review tasks depend on a developer's ability to build an accurate mental model of the code they are working with. Developers spend between 58% and 70% of their time on this activity [15, 20], and code that is difficult to understand slows development, increases the risk of introducing bugs, and makes collaboration harder. Improving code comprehensibility is therefore a high-impact goal—but improving it requires measuring it reliably.

Most existing studies measure comprehensibility through controlled human studies, which require careful design, participant recruitment, and often months of preparation. This makes it infeasible to rely on human studies whenever we need a comprehensibility estimate—for instance, to compare two implementations, to guide automated refactoring, or to evaluate LLM-generated code for human understandability. What we need instead is a reliable *automatic* proxy for code comprehensibility: a program analysis or metric that estimates comprehension difficulty directly from the code, without requiring human participants. Prior proposals—cyclomatic complexity [5], Halstead measures [4], lines of code [16]—turn out to correlate weakly, if at all, with developer comprehension behavior [8, 18, 21]. No reliable automatic proxy for code comprehensibility currently exists.

These prior proposals are surface-level and syntax-based: they capture structural properties of code without reasoning about its meaning or behavior. A natural next step is to ask whether semantic information can do better. One promising direction is code *verifiability*. The hypothesis is that code that is easier for verification tools to analyze is also easier for humans to understand. Prior work [9] investigated this hypothesis in a meta-analysis, operationalizing verifiability as the number of false positive warnings produced by static analysis tools, and found associations between verifiability and comprehension measures across a collection of prior studies. However, because their analysis aggregated heterogeneous prior studies with different code sources, structures, and participant populations, they could not control for confounders or establish a causal relationship. We want to re-evaluate this relationship in a controlled setting where verifiability is the only factor that varies between code snippets.

To do this rigorously, we need a reliable human-study baseline to test verifiability against: we need to know which code is actually harder to understand from a human study. This is where we encountered a first obstacle. The human-study proxies used in the literature—observable tasks assigned to participants (e.g., predicting program output) paired with measurements of their performance (e.g., response time or correctness)—have themselves never been systematically validated. Moreover, the field lacks a widely accepted definition of comprehensibility, so researchers often define it in terms of whatever proxy they happen to use [25]. Without knowing which human-study proxies are reliable, we cannot use them confidently to evaluate a candidate automatic proxy.

We therefore addressed this obstacle first. We introduce a principled conceptualization of comprehensibility, construct

an expert-consensus ground truth, and evaluate the reliability of commonly used human-study proxies against it. With that validated foundation in hand, we then proceed to test verifiability as a candidate automatic proxy in a controlled human study.

2 Phase 1 (Completed): Reconceptualizing Comprehensibility and Evaluating Proxy Reliability

We addressed the first obstacle—the lack of validated human-study proxies—in our recent work [1]. Our goal was to compare the reliability of commonly used human-study proxies against a ground truth measure of comprehension difficulty. The major challenge was that no such ground truth existed: there was no independent, principled measure of which code is actually harder to understand that we could use to evaluate the proxies against.

To construct one, we conceptualized code comprehensibility as expert agreement on relative comprehension difficulty. Using the Delphi protocol [6, 7, 12]—a structured method for eliciting and refining expert judgment under uncertainty, successfully applied in medicine [10, 14, 19, 22], geopolitical forecasting [11, 24], and other domains [13, 23]—a panel of five senior professional software engineers iteratively ranked eight real-world Java methods by comprehension difficulty and resolved disagreements through anonymized written feedback over three rounds.

We then conducted a controlled study in which 44 undergraduate CS students completed tasks corresponding to 14 comprehension proxies from the literature on the same eight methods, covering four task types (program function, input–output reasoning, syntax identification, and subjective rating) and three measure types (response time, answer correctness, and Likert-scale ratings). Correlating each proxy against the expert-consensus ranking via Kendall's τ revealed a clear hierarchy:

- **Most reliable:** time-based input–output proxies: (*time_output*, $\tau = 0.86$; *time_correct_output*, $\tau = 0.79$; *time_function*, $\tau = 0.71$).
- **Moderately reliable:** subjective difficulty ratings and general reading time ($\tau \in [0.50, 0.64]$).
- **Unreliable:** all syntax-based proxies (*acc_syntaxBL*, $\tau = -0.37$; *time_syntaxBL*, $\tau = -0.21$), showing near-zero or negative correlations regardless of whether time or correctness is used as the measure.

These results have two consequences. First, they provide a validated set of proxies for future human studies: researchers can now confidently use input–output response time as a reliable approximation of comprehension difficulty. Second, they call into question prior work that relied on syntax-based proxies, whose negative correlation with expert judgments suggests they measure a fundamentally different construct than comprehension difficulty.

3 Phase 2 (Proposed): Verifiability as an Automatic Proxy

With a validated human-study baseline in hand, we can now rigorously test candidate automatic proxies. The first candidate we investigate is code *verifiability*: the ease with which static verification tools can analyze code. This is motivated by a widely held assumption in the verification community—captured explicitly in the Checker Framework manual [3], which advises developers to "rewrite your code to be simpler for the checker to analyze; this is likely to make it easier for people to understand, too"—that more verifiable code is also more comprehensible.

A prior meta-analysis [9] found associations between false positive warnings produced by static analysis tools and comprehension measures. However, because it aggregated heterogeneous prior studies, it cannot isolate verifiability as the cause: the snippets differ in source, structure, size, and—as Phase 1 shows—proxy quality. Our proposed study addresses all of these limitations.

Treatment. Verifiability is operationalized as a binary condition relative to a fixed set of Checker Framework checkers [17]. *High-verifiability* code produces no false positive warnings; *low-verifiability* code is a semantically equivalent version of the same method that produces one or more false positives under the same checkers. Paired variants are constructed via semantics-preserving transformations (defined as input–output equivalence) in two phases: first by eliminating false positives from warning-producing methods, then by injecting false positives into warning-free methods. Transformations span a range of code-length changes to avoid collinearity between verifiability condition and code length.

Outcome and design. The primary outcome is response time on input–output tasks—the most reliable proxy identified in Phase 1. The experiment uses a mixed design: within-subjects over snippets and between-subjects over verifiability condition, so no participant sees both versions of the same method. Latin square counterbalancing ensures balanced exposure to both conditions across participants. The analysis uses linear mixed-effects models with a fixed effect for verifiability condition, random intercepts for participants and snippets, and covariates for snippet order, code length, and transformation type.

Interpretation. If verifiability alone explains comprehension difficulty well, it can be used as a standalone automatic proxy. If it only partially explains comprehension difficulty, it becomes one signal among several in a combined automatic proxy. A null result is equally meaningful: it would provide controlled evidence that the associations in prior work [9] were driven by confounds rather than a genuine relationship.

4 Evaluation and Future Directions

Phase 2 evaluation. The primary evaluation metric is the fixed effect of verifiability condition on response time on input–output tasks. The main threats to validity are: *internal*—transformations may introduce residual structural differences beyond verifiability, mitigated by simple, manually validated transformations and structural covariates in the model; *construct*—false positive warnings and response time are each one-dimensional operationalizations of broader constructs; and *external*—results are scoped to Java, the Checker Framework, and undergraduate participants, and generalization to other languages, tools, and professional developers is future work. All data and analysis scripts will be made publicly available.

Broader agenda. Beyond Phase 2, the research agenda includes: (1) evaluating whether LLM-based agents can reliably substitute for human participants in comprehension studies, enabling further scaling of proxy validation; (2) systematically reassessing prior comprehensibility studies through the lens of validated proxies, identifying which findings remain robust and which relied on unreliable measurements; (3) investigating how different forms of accidental complexity [2] affect comprehension time, using input–output response time as the validated outcome; and (4) developing tools that identify hard-to-understand code and suggest semantics-preserving refactorings, contingent on establishing a reliable automatic proxy.

Together, this work aims to establish a principled and empirically grounded foundation for measuring and improving code comprehensibility.

References

[1] Erfan Arvan, Nadeeshan De Silva, Oscar Chaparro, and Martin Kellogg. 2026. On the Reliability of Code Comprehension Proxies. *arXiv preprint arXiv:2605.23008* (2026).

[2] Frederick Brooks and H Kugler. 1987. *No silver bullet.* April.

[3] Checker Framework Developers. 2024. *Checker Framework Manual.* https://checkerframework.org/manual/ Section 2.4.5: What to do if a checker issues a warning about your code.

[4] Shyam R Chidamber and Chris F Kemerer. 1994. A metrics suite for object oriented design. *IEEE Transactions on software engineering* 20, 6 (1994), 476–493.

[5] Bill Curtis, Sylvia B. Sheppard, Phil Milliman, MA Borst, and Tom Love. 2006. Measuring the psychological complexity of software maintenance tasks with the Halstead and McCabe metrics. *IEEE Transactions on software engineering* 2 (2006), 96–104.

[6] Norman Dalkey and Olaf Helmer. 1963. An experimental application of the Delphi method to the use of experts. *Manage. Sci.* 9, 3 (1963), 458–467.

[7] Norman C. Dalkey. 1969. *The Delphi method: an experimental study of group opinion.* RAND Corp., Santa Monica, CA. doi:10.7249/RM5888

[8] Janet Feigenspan, Sven Apel, Jorg Liebig, and Christian Kastner. 2011. Exploring software measures to assess program comprehension. In *2011 International Symposium on Empirical Software Engineering and Measurement.* IEEE, 127–136.

[9] Kobi Feldman, Martin Kellogg, and Oscar Chaparro. 2023. On the relationship between code verifiability and understandability. In *ESEC/FSE.* 211–223.

[10] Ileana Gefaell Larrondo et al. 2026. Strengthening Primary Health Care in Europe: A Delphi study towards accessibility, equity and continuity of care. *Eur. J. Gen. Pract.* 32, 1 (2026), 2619226.

[11] Oleksandra Ishchenko et al. 2025. Barriers and opportunities for Demand Response Aggregation in Ukraine and Norway: A Delphi-based study. *Energy* 328 (2025), 136296.

[12] Dmitry Khodyakov, Sean Grant, Jack Kroger, and Melissa Bauman. 2023. *RAND methodological guidance for conducting and critically appraising Delphi panels.* RAND Corp., Santa Monica, CA. doi:10.7249/TLA3082-1

[13] Brady D Lund. 2020. Review of the Delphi method in library and information science research. *J. Doc.* 76, 4 (2020), 929–960.

[14] Sarah B Maness, Stacey B Griner, and Erika L Thompson. 2025. Expert Consensus on Indicators of Social Determinants of Health: A Modified Delphi Study. *J. Prim. Care Community Health* 16 (2025).

[15] Roberto Minelli, Andrea Mocci, and Michele Lanza. 2015. I know what you did last summer: an investigation of how developers spend their time. In *ICPC.* 25–35.

[16] Alberto S Nuñez-Varela, Héctor G Pérez-Gonzalez, Francisco E Martínez-Perez, and Carlos Soubervielle-Montalvo. 2017. Source code metrics: A systematic mapping study. *Journal of Systems and Software* 128 (2017), 164–197.

[17] Matthew M. Papi, Mahmood Ali, Telmo Luis Correa Jr., Jeff H. Perkins, and Michael D. Ernst. 2008. Practical pluggable types for Java. In *ISSTA 2008, Proceedings of the 2008 International Symposium on Software Testing and Analysis.* Seattle, WA, USA, 201–212. doi:10.1145/1390630.1390656

[18] Norman Peitek, Sven Apel, Chris Parnin, André Brechmann, and Janet Siegmund. 2021. Program comprehension and code complexity metrics: An fmri study. In *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE).* IEEE, 524–536.

[19] Jennifer Roggenbuck, Breda HF Eubank, Joshua Wright, Matthew B Harms, Stephen J Kolb, and the ALS Genetic Testing and Counseling Guidelines Expert Panel. 2023. Evidence-based consensus guidelines for ALS genetic testing and counseling. *Ann. Clin. Transl. Neurol.* 10, 11 (2023), 2074–2091.

[20] Simone Scalabrino, Gabriele Bavota, Christopher Vendome, Mario Linares-Vásquez, Denys Poshyvanyk, and Rocco Oliveto. 2017. Automatically assessing code understandability: how far are we?. In *ASE.* 417–427.

[21] Simone Scalabrino, Gabriele Bavota, Christopher Vendome, Mario Linares-Vasquez, Denys Poshyvanyk, and Rocco Oliveto. 2019. Automatically assessing code understandability. *IEEE Transactions on Software Engineering* 47, 3 (2019), 595–613.

[22] Zhida Shang. 2023. Use of Delphi in health sciences research: a narrative review. *Medicine* 102, 7 (2023), e32829.

[23] Erin A Taylor et al. 2024. Assessing the feasibility and likelihood of policy options to lower specialty drug costs. *Health Aff. Scholar* 2, 10 (2024), qxae118.

[24] Abbie Tingstad et al. 2025. Divergent trajectories of Arctic change: Implications for future socio-economic patterns. *Ambio* 54, 2 (2025), 239–255.

[25] Marvin Wyrich, Justus Bogner, and Stefan Wagner. 2023. 40 years of designing code comprehension experiments: a systematic mapping study. *ACM Comput. Surv.* 56, 4 (2023), 1–42.